PRISM

(Programmable Reduction & Inference Search Manager)

Programmable Search Control for Hyperon

Project Proposal — AGI Engineering Competition

Track A: Hybrid AGI — Cognitive, Neuro-Symbolic & Hyperon-Based AGI

iCog Labs

Github Repo

Prototype Results(POC)

Author: *Abel Tesfa*

1. Executive Summary

Hyperon provides ways to represent knowledge, apply rules, reduce expressions, and pursue goals. These capabilities create a recurring problem: at any given moment, a component may have many possible operations it could perform, but limited time and memory to explore them. The rules of a component determine *what is permitted*; they do not, by themselves, determine *which permitted possibility is worth exploring next*.

PRISM (the Programmable Reduction & Inference Search Manager) is a proposed search-control layer for that problem. It would let a Hyperon component expose candidate operations and their outcomes through a defined interface. PRISM would then filter candidates, rank them against an objective, manage the search frontier and its budget, detect stalled exploration, and select a strategy suited to the component. The underlying component would remain responsible for executing operations and determining their meaning. PRISM would control the allocation of search effort, not replace the component's logic. Some of the hyperon components that PRISM could help are PLN, Openpsi and MOSES. We have performed some experiments on PLN and Openpsi.

Two existing integrations make this more than an abstract generality:

- **PLN is the inference reference backend.** Probabilistic Logic Networks supply the rules, truth-value calculations, and evidence handling. PRISM guides which inference tasks and premises to examine and can manage a goal-directed search over PLN-derived transitions. A conclusion must still come from PLN's own rule application.
- **The iCog PeTTa-OpenPSI port is the planning reference backend.** OpenPsi supplies applicable actions and successor states from its own rule space. PRISM selects among those actions and searches for a goal-reaching plan under a budget. OpenPsi retains responsibility for action execution, demands, affect, and feedback.

The two integrations address the same *control question* but not the same task. PLN searches for justified conclusions; OpenPsi searches for applicable action sequences. They therefore test whether PRISM can expose shared control concepts—candidate selection, goal relevance, frontier management, budgets, fallback, and verification—without assuming that every Hyperon component is a theorem prover or must use A*.

The initial release would consolidate these reference adapters into a defined, reproducible control interface and establish how their benefits, overhead, and failures are measured. Subsequent quarters would extend the interface to additional workloads, strengthen MeTTa-native operation, and investigate which control policies transfer between components. The project is the reusable control layer and its research program; PLN and OpenPsi are its first two testbeds.

2. Introduction & Motivation

Hyperon's AtomSpace can store structured knowledge, but storing facts alone is not reasoning. An AGI-relevant system must derive new knowledge, consider possible actions, and direct limited computation toward useful goals. Hyperon has components for these activities; what remains difficult is choosing among the many operations they could perform at any moment.

PLN makes this problem concrete for inference. Its rules can derive conclusions from uncertain premises, but a query may admit many legal rule applications and premise combinations; the system must decide which inference tasks merit its limited budget. OpenPsi presents the corresponding challenge in goal-directed behavior: an applicable action may look promising locally yet lead to a dead end. MOSES presents a third form of the problem in program search: among many candidate programs and possible variations, it must

decide which to evaluate and explore further. These workloads differ, but each must allocate limited computation among competing possibilities.

PRISM targets this **control problem**, not the underlying logic or action semantics. It is a programmable layer for filtering and ranking candidates, managing alternatives across a search, allocating budgets, and deciding when to stop or change strategy. PLN remains responsible for inference rules, truth values, and evidence. OpenPsi remains responsible for applicable actions, state transitions, and action execution. PRISM's proposed standalone contribution is an interface and set of control policies that can serve both backends—and eventually other Hyperon components—without treating every workload as the same kind of search.

This is infrastructure work inside Hyperon's reasoning and planning processes, not merely an application built on top of them. PLN is the first inference reference backend; the iCog PeTTa-OpenPSI port is a second, distinct planning reference backend. Their differences are central to the research: a reusable controller must express what they share while preserving what each one does differently.

3. Background

3.1 The AtomSpace and MeTTa

The AtomSpace is Hyperon's typed knowledge hypergraph; MeTTa is the language used to write, type-check, and reduce (evaluate) expressions against it. Everything is represented as typed atoms in this store and this is where Hyperons components act.

3.2 The Hyperon components PRISM aims to support

PRISM is intended for any Hyperon component that generates multiple possible next steps and must decide which to explore with limited time or memory. Those steps might be inferences, actions, program variations, or expression reductions. The component remains responsible for defining valid steps and their outcomes

3.3 Why the hard part is search, not logic

A component may define exactly how to perform an operation without knowing which operation is worth performing next. A single state can offer many valid inferences, actions, program variations, or reductions. Each choice may create further choices. Most may be valid but unhelpful for the current objective, while a promising-looking choice may lead to a dead end.

Simply increasing the time or step budget can allow more unproductive exploration. Effective search requires decisions across the *whole process*: which candidates to examine first, which alternatives to retain, which states have already been visited, how much effort to spend on each branch, and when to stop or change strategy. The best policy may differ by component and task; control itself also has a computational cost that must be justified by its results.

PRISM addresses these decisions while leaving the meaning and execution of each step to the component that produced it. That separation can preserve the validity of results, but it does not guarantee that a useful path will be found: filtering or a tight budget may discard one. Result validity, search success, and search efficiency must therefore be measured separately.

3.4 What the industry has tried

- Static/heuristic rule ordering — hand-tuned priority lists; brittle, does not generalize, requires constant manual retuning as the AtomSpace grows.
- Exhaustive or breadth-limited search — correct but does not scale; combinatorics dominate quickly.
- Learned premise/tactic selection in classical theorem provers (e.g. graph-neural or transformer-based rankers guiding Lean/Isabelle/Coq-style proof search) — validates the general

pattern of a learned model ranking proof-search moves, but built for crisp, non-probabilistic logics, with no connection to an AtomSpace/MeTTa representation.

- ECAN — spreads "importance" across atoms generally (Short/Long-Term Importance, Hebbian learning), which is valuable but is not a proof-search-specific mechanism; it does not rank inference steps, only atom salience, while valuable it is not by itself a policy for selecting complete inference steps, retaining alternative proof paths, or verifying a goal-reaching plan.

3.5 Existing prior art and current feasibility

PRISM builds on the idea that a system must decide where to spend limited computation, not just define which operations are possible. Geisweiller's MeTTa inference-control work demonstrates programmable pruning of search branches. Rational metareasoning provides a broader basis for weighing the benefit of a computation against its cost. Work such as CrossBeam shows that guided search is relevant beyond logical inference, including program synthesis.

PRISM does not claim to invent search control. Its proposed contribution is a reusable control interface and evaluation framework for Hyperon components: different backends retain their own rules and outcomes, while PRISM manages how their candidates are explored under a budget.

This is feasible with the current runtimes. PRISM already has working reference integrations with PLN and the iCog PeTTa-OpenPSI port, demonstrating the approach in inference and action planning. MOSES is a proposed future test for program search; no integration or result is claimed for it yet. The remaining research question is whether the shared interface provides a measurable benefit across these different workloads without imposing more overhead or restricting what their native search can do.

4. Problem Statement & Gaps

Hyperon components can often generate more valid next steps than they can afford to explore. A reasoning system may have many rule applications available; a planner may have several applicable actions; a program-search system may have many candidate variations. The component defines these possibilities, but deciding which to investigate, how long to pursue them, and when to try an alternative is a separate problem.

This is not a claim that these components lack search strategies. PLN prioritizes inference tasks, and OpenPsi has native planning and action selection. But their existing strategies proved insufficient on several evaluated workloads: in a noisy PLN chain, native search solved 0% of runs while PRISM-guided search solved 100% under the same step budget; on 21 goals from one OpenPsi conversation-rule graph, a matched native v2-selector traversal reached 3, while PRISM reached 20. These results show a concrete need for stronger control in the tested settings, alongside a broader architectural gap: Hyperon lacks a reusable way to expose search choices, apply alternative policies, and compare their benefits and costs across components.

Gap	What PRISM adds
Search choices are exposed and managed differently by each component.	A common adapter interface through which a backend can expose candidates, goals, costs, transitions, and results without giving PRISM ownership of their semantics.
A local ranking decision may lead to a dead end several steps later.	A search manager that can retain alternatives, track explored states, allocate a budget, and switch strategies when appropriate.
Static/exhaustive search does not scale with AtomSpace size	Best-first / beam search using learned usefulness scores, spending compute on promising steps only

Gap	What PRISM adds
Candidate relevance cannot always be captured by one fixed signal.	Pluggable policies that can consider goal relevance, structure, uncertainty, estimated cost, and search history when those signals exist for the backend. No single score is assumed to fit every workload
Broader exploration can improve success but also introduce substantial overhead.	Explicit measurement of success, search work, latency, memory use, and cold-start cost against each backend's native strategy.
Promising guidance on one workload may not transfer to another.	Separate adapters and evaluations across inference and planning first, with program search as a proposed further test—not an assumed generalization result.
A controller could accidentally blur the boundary between guidance and execution.	Backend-owned application and verification: PRISM may propose or select a step, but the backend must determine its valid outcome.

4.1 Prioritization Rationale (Stage 3)

Criterion	Assessment
Criticality	High. Many AGI-oriented components face branching choices under limited computation. Better control could increase the number of useful conclusions, plans, or programs found within a practical budget. The importance of programmable inference control is also established in Geisweiller's MeTTa work.
Research Gap	The individual ideas—heuristics, learned guidance, pruning, and budgeted search—are established. The open question is whether a reusable Hyperon control interface can deliver measurable benefits across components with different operations and search spaces.
Novelty	Moderate-High, PRISM's proposed novelty is the combination of a shared backend contract, programmable policies, backend-owned verification, and cross-component evaluation. It should not claim to be the first learned search controller or the first instance of inference control in Hyperon.
Feasibility	High and now substantially confirmed, PRISM has working reference integrations with PLN and the iCog PeTTa-OpenPSI port. They show that control can be inserted while the respective backends supply their own inference results or action transitions. A MOSES adapter remains proposed work, and the existing integrations do not yet have identical feature coverage.
Impact	Potentially High, but conditional. An improvement benefits a component only when its search bottleneck is significant and PRISM's guidance saves more work than it costs. Simple tasks may favor native control.
Strategic Relevance	High for a standalone Hyperon project: the deliverable is a reusable search-control capability and evaluation platform, not just a change to one engine's priority function.

4.2 AGI Relevance

An AGI-oriented system cannot exhaustively explore every inference, plan, or program candidate it generates. It must decide where to spend computation, preserve promising alternatives, recognize unproductive paths, and act within time and memory limits. Search control is therefore a core capability: it determines how effectively a system uses the knowledge and operations it already has.

PRISM develops this capability as reusable Hyperon infrastructure. Its PLN and iCog PeTTa-OpenPSI integrations demonstrate applications to inference and planning; a future MOSES integration would extend the research to program search, using policies suited to that workload. By making search effort programmable and measurable across components, PRISM aims to help Hyperon turn available computation into more useful results.

5. System Overview — What PRISM Actually Is

Imagine a Hyperon component working toward an objective. At any moment, it may have several possibilities to explore: an inference, an action, a program variation, or an expression reduction. The component knows how to perform these operations, but deciding which possibility is worth limited computation is a separate problem.

PRISM is a programmable, ultimately learned search-control layer for that problem. It filters candidates, estimates their usefulness, manages alternatives across multiple steps, and allocates the search budget. Its central research direction is a neural policy trained from search experience: rather than relying indefinitely on hand-written priorities, PRISM aims to learn which patterns of states, goals, candidates, and past outcomes predict productive exploration.

The analogy: the underlying component is the specialist doing the work; PRISM is the coordinator deciding where that specialist should spend effort. At first, the coordinator can use explicit rules. As it collects examples of successful and failed searches, a neural model can learn better judgments about what to try first, what to keep as an alternative, and when a branch is unlikely to pay off. The specialist still determines what each operation means and whether it succeeds.

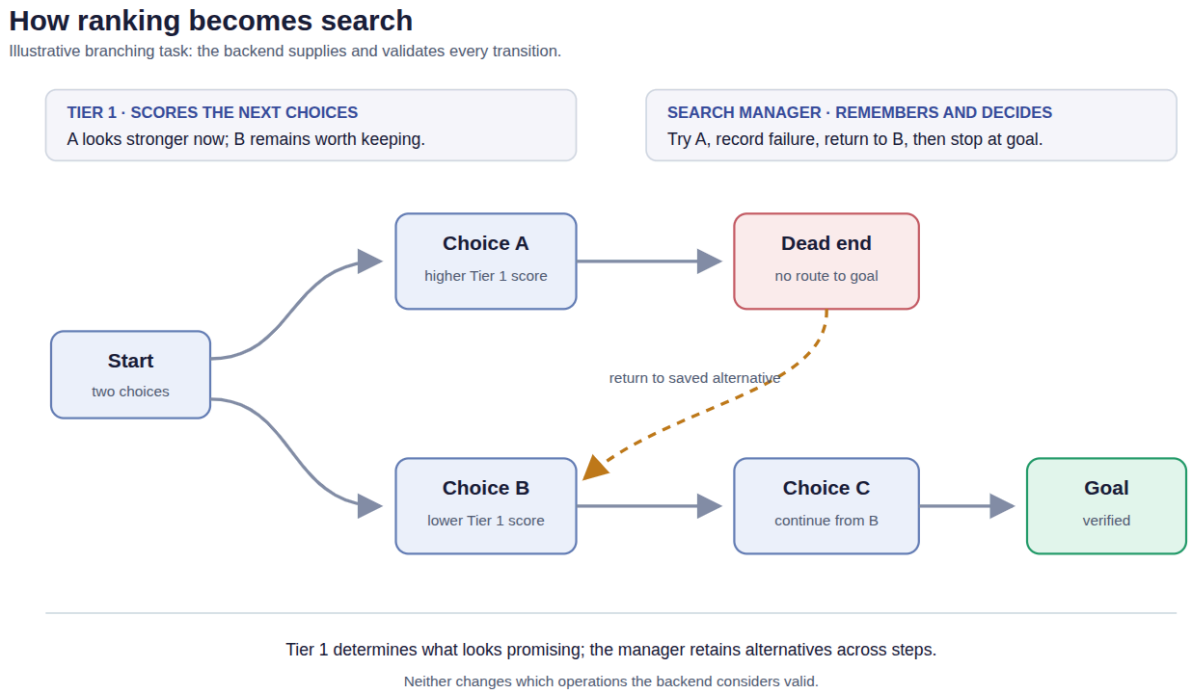
This makes PRISM more than a neural scorer attached to one component. The proposed system combines a shared control interface, a search manager, and trainable policies. Different backends may require different candidate representations or search strategies; the research question is how much useful control knowledge can be learned and transferred between them. PLN and the iCog PeTTa-OpenPSI port are the current reference integrations. MOSES is a proposed future test for program search.

What PRISM is not: it does not replace a component's rules, invent a successful result, or decide that an unsupported conclusion is true. A neural policy may recommend a candidate or subgoal, but the backend must still apply the operation and verify the outcome. Learning changes where PRISM directs effort, not the authority that determines what happened.

5.1 How PRISM works, step by step

- **Expose candidates:** A backend supplies possible next operations and information about its current state and objective. It remains responsible for checking applicability and carrying out a selected operation.
- **Filter candidates:** PRISM can remove clearly irrelevant options before more expensive scoring or execution. This stage is configurable because filtering too aggressively could discard a useful path.
- **Score with a control policy:** PRISM ranks the remaining candidates. A fast symbolic policy provides a reliable starting point. The proposed neural policy would learn from search traces—candidate features, choices, costs, successes, failures, and their longer-term consequences—to predict which options deserve attention.
- **Manage the search over time:** A high score only says what looks promising at one moment. The search manager retains alternatives, tracks explored states, allocates computation, handles dead ends, and chooses a strategy suited to the backend.
- **Apply and verify through the backend:** PRISM requests a selected operation; the backend determines its outcome. A suggested subgoal or promising neural prediction is not treated as an achieved result until the backend establishes it.

- **Learn and improve:** Outcomes become evaluation data and, where appropriate, training data. Neural policies are compared against symbolic policies and native backend strategies on held-out workloads, including unfamiliar structures—not judged solely by accuracy on examples resembling their training traces.



Stated plainly: PRISM begins with working programmable search control and develops toward neural control learned from experience. Its purpose is to learn where computation is most valuable while each Hyperon component retains authority over its own operations and results.

6. Technical Approach

The technical approach focuses primarily on PLN, where PRISM's integration and search mechanisms are most developed. It also examines the iCog PeTTa-OpenPSI port to show how the same control architecture applies to action planning, with backend-specific adaptations. MOSES is discussed as a potential extension to program search, not as an existing integration.

6.1 Grounding the hook in the real implementation

PRISM's PLN integration begins at an existing control point in `lib_pln.metta`. `PLN.Derive` maintains inference tasks and beliefs, selects a task, applies PLN rules, and adds the resulting conclusions to its bounded search state. Its original `PriorityRank` function ranks tasks by truth-value confidence. This can favor a highly confident task that is unrelated to the current query, spending limited derivation steps on distractors while a more useful path remains unexplored. The limitation is in which inference receives attention, not in PLN's inference rules or truth-value formulas.

PRISM adds `PriorityRankGoal` to score tasks against the query goal. `PLN.Query` passes that goal into `PLN.Derive`, while calls without a goal remain supported. If no goal is available or scoring fails, selection falls back to confidence rather than halting derivation. A separate `PremiseFilter` hook narrows the beliefs considered with the selected task and falls back to the full belief buffer if filtering fails. Together, these hooks guide task and premise selection without changing what PLN's rules derive.

The current scoring and filtering functions run in Python through a PeTTa–Prolog–Janus bridge. Candidate–goal scores are cached to avoid repeated calculation, although a cache hit can still incur a bridge

call. Goal-aware `PriorityRankNegGoal` and `LimitSizeGoal` functions also exist, but the current `PLN.Derive` path still uses confidence-based `LimitSize` queue trimming. PRISM therefore guides task selection and premise filtering in that loop; it does not yet control every queue decision.

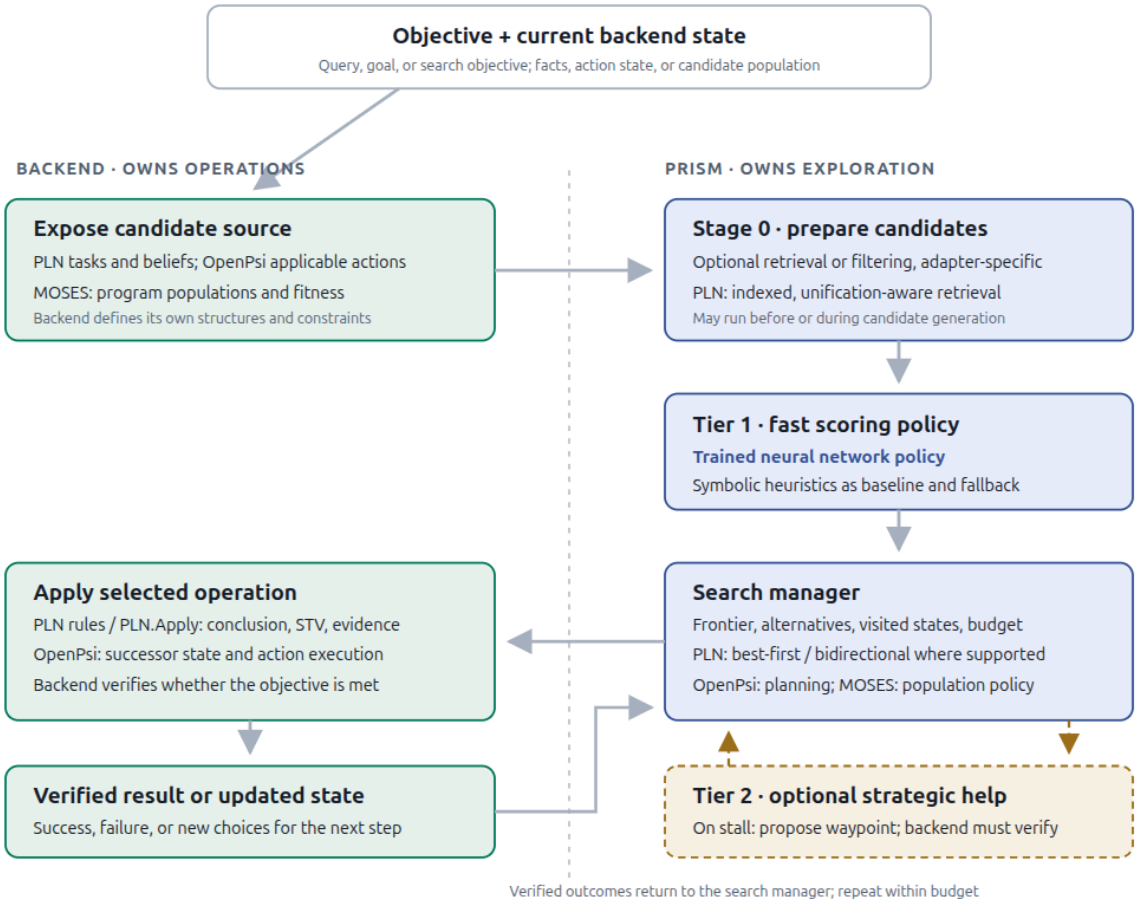
For explicit multi-step search, PRISM uses `PLN.Apply` as the execution boundary. The search manager chooses which premise pair to explore, but the live MeTTa PLN implementation applies the rule and returns the conclusion, truth value, and evidence stamp.

6.2 A staged guidance architecture: indexed pre-filtering + two-tier scoring (not a per-candidate LLM call)

Calling a large language model for every possible next step would make search prohibitively expensive. Cost can arise even earlier: a component may generate a large number of candidates before any scorer runs. PRISM therefore separates frequent, inexpensive control decisions from occasional, more expensive guidance. **An LLM is not called for every candidate.**

- **Stage 0 — candidate filtering:** Before detailed scoring, PRISM uses information supplied by the backend to narrow the options worth considering. In the current PLN integration, a concept-based index selects beliefs that share relevant concepts with the active task or goal, reducing the premise pairs sent to PLN's rules. This is **not** full MeTTa unification or a guarantee that every useful premise will be retained. Filtering must be evaluated for both saved work and missed paths, with a safe fallback when it cannot operate. Other backends may expose different, more suitable ways to filter candidates.
- **Tier 1 — fast candidate policy:** Candidates surviving Stage 0 receive inexpensive scores that guide their order of exploration. The current symbolic PLN policy combines goal overlap, confidence, and a depth-related bonus. This provides a working baseline without training data. A central goal of the project is a **neural Tier 1 policy** trained on search traces to predict which choices are productive beyond the immediate step. The initial neural experiment establishes neither an improvement over the symbolic baseline nor robust generalization; developing and testing a model that does is part of the proposed research. A Tier 1 score guides search—it does not determine whether a backend operation is valid.
- **Tier 2 — occasional strategic guidance:** When ordinary search stalls or needs a useful intermediate objective, PRISM can request higher-level guidance from a language model. This is invoked at selected points, not on each candidate or each step. In the current PLN use case, a suggested subgoal becomes something for PLN to establish from existing premises, **not a new fact asserted by the model**. Extending strategic guidance to another backend would require defining what a valid subgoal means for that backend and how its achievement is checked.

These stages reduce different costs: Stage 0 limits the candidates considered, Tier 1 orders the remaining choices quickly, and Tier 2 offers strategic help when the expected benefit warrants its expense. The search manager, described next, uses their outputs across multiple steps while tracking alternatives, budgets, and outcome



6.3 A formal search algorithm, not "best-first" left vague

A useful candidate score is not, by itself, a search strategy. PRISM's search manager decides which candidates to explore across multiple steps, which alternatives to retain, how to avoid repeated work, and when to stop under a time or memory budget. Tier 1 scores choices; the search manager turns those scores into a sustained exploration policy.

For PLN proof search, the current demonstration uses an A*-style best-first agenda with $f(n) = g(n) + h(n)$. The accumulated cost $g(n)$ reflects steps taken and confidence degradation; $h(n)$ uses Tier 1's estimate of goal relevance. This is a practical ranking function, **not a claim of optimal A***: the present heuristic is not proven admissible. Beam limits and visited-state tracking keep exploration bounded.

The proposed project makes the policy programmable rather than prescribing A* everywhere. Proof graphs may benefit from best-first or bidirectional exploration; action planning may require policies sensitive to execution cost and failure; a future MOSES adapter would likely allocate effort across a population of programs. A learned policy—and potentially a learned value estimate—would become a central part of this manager as training data and evaluation mature. MCTS or other policies are research options, not capabilities claimed by the current demo.

6.4 Forward, backward, and strategic exploration

Forward search asks what can be produced from the current state. Backward search asks what would need to be established for the goal to become reachable. PRISM can use both when a backend exposes enough structure to do so, retaining separate frontiers and looking for a valid connection between them.

The PLN demonstration supports forward search and a restricted form of backward decomposition for selected rule and relation types. A proposed connection is not accepted merely because two terms appear to

match: the resulting derivation must be replayed and checked through PLN's own rule application. Extending backward search to more of PLN's rule repertoire is part of the proposed work.

Tier 2 has a different role. When ordinary search stalls, a language model may suggest a waypoint, subgoal, or change of tactic. That suggestion is **search guidance, not a new fact or a valid operation by decree**. PRISM returns it to the relevant backend and search process for establishment or rejection. Backward search therefore does not depend on an LLM, and an LLM does not replace backend verification.

6.5 Candidate representation

PRISM needs a common way to describe a choice without pretending that an inference step, an action, and a candidate program are the same object. An adapter exposes the search objective, a backend-native candidate handle, relevant state and history, available structural features, estimated cost or uncertainty, and the operation that asks the backend to apply or evaluate the choice.

For example, a PLN adapter can describe a rule and its premises, including truth values and evidence stamps, while preserving the original MeTTa terms for execution. An OpenPsi adapter can describe an applicable action, its preconditions, and the successor state produced by OpenPsi. A future MOSES adapter would expose program candidates and fitness-related information instead.

Tier 1 consumes structured features to produce a fast numeric score. The demonstration begins with a symbolic scorer; developing and testing a neural policy that generalizes beyond hand-designed examples is a major project objective. Tier 2 may receive a readable summary for occasional strategic reasoning. Neither tier calculates PLN truth values or determines whether an OpenPsi transition is valid.

```
;; Illustrative candidate description
Rule: deduction


Premises: (Inheritance Alice Ancestor-of-Bob) [0.9, 0.8]
          (Inheritance Bob Ancestor-of-Carol) [0.85, 0.9]


Goal: (Inheritance Alice Ancestor-of-Carol)


→ Tier 1: assigns numeric score from the trained scorer's forward pass over these features
   (a GNN model computation, not an LLM estimating relevance or confidence from text)

→ Search manager: decides when to try this candidate relative to other

                paths and the remaining budget.

→ PLN: applies the rule and computes the conclusion, truth value, and

     evidence using its own implementation.


→ Tier 2 (only if stalled): does this goal need decomposing into subgoals?
```

6.6 Two ranking modes

PRISM supports two distinct search objectives:

- **Goal-directed control:** allocate effort toward a specified query, target state, or desired result. This is the primary mode demonstrated in the current PLN and OpenPsi work.
- **Exploratory control:** allocate effort toward useful discoveries when there is no single target—for example, novel high-confidence conclusions or promising programs under a stated utility measure and budget.

Exploratory control is not simply goal-directed scoring with the goal removed. It needs a backend-appropriate definition of usefulness and separate evaluation for novelty, quality, and computation spent. Developing that mode is part of the broader proposal, not a result already established by the demo.

6.7 Engineering stack

- **Backend adapters:** A common interface exposes available candidates, accepts a selected operation, and returns its outcome. PLN supplies inference steps and their truth values; OpenPsi supplies applicable actions and successor states. A MOSES adapter will investigate whether the same control framework can guide program search.
- **Stage 0 — candidate preparation:** Backend-specific retrieval and filtering reduce the choices that require scoring. For PLN, this includes indexes over structural and unification patterns to retrieve premises relevant to a goal or subgoal. Other backends can provide their own indexing or candidate-generation methods through the same interface.
- **Tier 1 — fast neural policy:** A compact neural network scores candidates using their structure, context, uncertainty, and search history. Symbolic heuristics provide an initial policy and a dependable fallback. Training uses search traces, with held-out tasks and unfamiliar structures used to test whether the learned policy generalizes.
- **Search manager:** A programmable controller maintains frontiers, alternatives, visited states, and computation budgets. It supports best-first and bidirectional proof search, goal-directed planning, and backend-specific policies where a path-based strategy is unsuitable. Tier 1 influences priority; the manager decides how exploration unfolds over time.
- **Tier 2 — strategic guidance:** An LLM is invoked selectively for stall recovery, goal decomposition, or a change of search tactic—not for every candidate. Its suggestions re-enter the normal search process and must be established through backend operations.
- **Backend execution and verification:** PRISM controls *which* operations receive attention, while each backend defines *what* those operations mean. PLN computes conclusions, truth values, and evidence; OpenPsi determines action applicability and successor states. PRISM cannot turn a suggested step into a valid result on its own.
- **Trace and evaluation pipeline:** The system records candidates, scores, decisions, outcomes, search paths, and compute costs. These traces support policy training and rigorous comparisons with each backend's native control strategy, including success under fixed budgets, solution quality, generalization, and control overhead.

7. Evaluation Plan

PRISM will be evaluated as a search-control system, not merely as a candidate scorer. The central question is whether it finds more valid results—or finds the same results with less computation—when compared with each backend's native strategy under matched tasks and budgets. PLN is the primary evaluation backend; OpenPsi tests whether the approach transfers to planning and this will continue with other hyperon components.

7.1 Domains

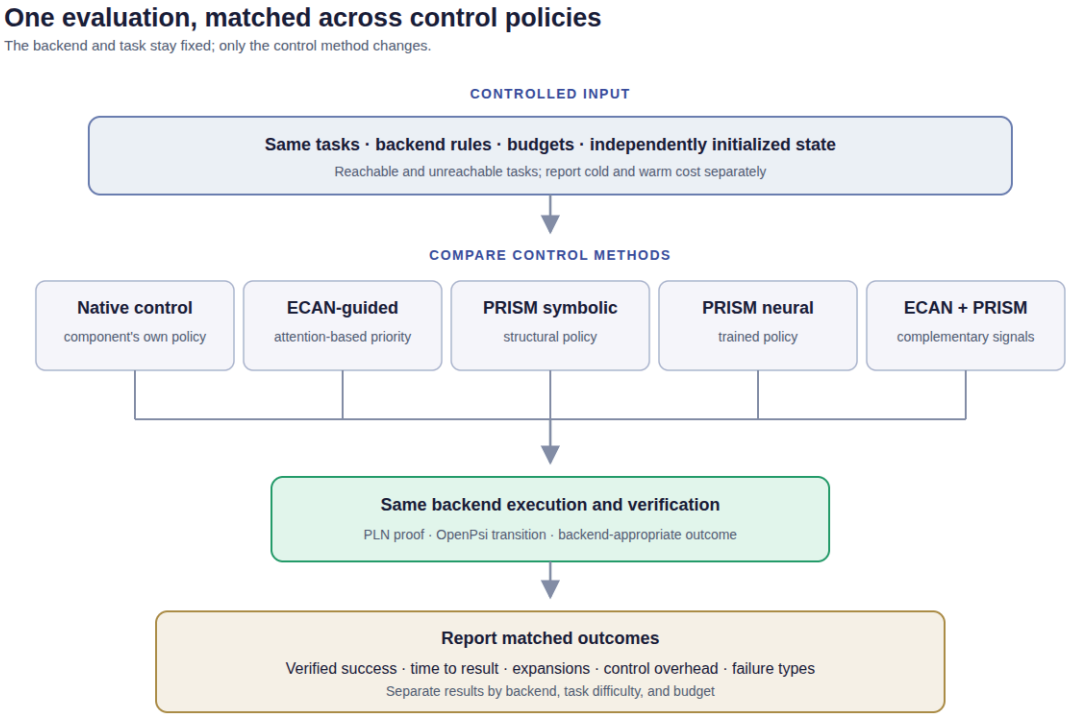
PLN's small repository examples are useful for correctness checks but are not sufficient evidence of search improvement. The reported evaluation will use:

- **Controlled inference graphs:** Synthetic chains and branching graphs with known solutions, varying proof depth, distractor count, cycles, and misleading high-confidence branches. These isolate why a policy succeeds or fails as search difficulty increases.
- **Real knowledge-graph workloads:** Multi-hop queries over imported knowledge graphs at progressively larger scales. The translation into MeTTa must preserve relation meaning and evidence provenance; an arbitrary graph path will not be counted as a valid PLN proof. Tasks will include both reachable and unreachable queries.
- **OpenPsi planning domains:** Multiple rule sets with different branching patterns, dead ends, and action costs. Plans will be replayed through OpenPsi's transitions. Where action execution or a controlled simulator is available, the evaluation will also measure actual goal completion under stochastic failures.
- **Generalization tests:** Train a neural Tier 1 policy on one set of graph structures or task families and evaluate it on held-out entities, unfamiliar structures, and distinct domains. Splits will prevent near-identical paths from appearing in both training and test data.
- **Program-search extension:** If the MOSES adapter is completed, evaluate it on held-out program-synthesis tasks using MOSES's own candidate evaluation and a policy suited to population search. This is an extension, not a prerequisite for the PLN and OpenPsi results.

7.2 Baselines

All methods will use the same tasks, backend rules, independently initialized states, and matched computation budgets.

- **Native component control:** Each backend's existing method for prioritizing inference, planning, or program-search work.
- **ECAN-guided control:** Use ECAN's atom-importance signals to prioritize candidates in each evaluated component. This tests whether general attention allocation is sufficient, or whether PRISM's goal-aware, multi-step search control adds value.
- **Unguided PRISM search manager:** Use the same frontiers, budgets, and duplicate handling as PRISM, but without informed candidate ranking. This isolates the benefit of search management itself.
- **PRISM with symbolic Tier 1:** Use fast structural heuristics as the policy baseline.
- **PRISM with neural Tier 1:** Use the trained policy under otherwise identical conditions. Ablations will measure the contributions of Stage 0, Tier 2, and individual feature groups.
- **ECAN + PRISM:** Test whether attention-based salience and search guidance are complementary rather than competing approaches.



7.3 Core metrics

- **Success under fixed budgets:** Percentage of queries with a backend-verified proof or plan at each step, time, and memory budget. Reachable and unreachable tasks will be reported separately.
- **End-to-end cost:** Wall-clock time, backend calls, candidate-generation time, scoring time, Tier 2 calls, and peak memory. Both cold-start and warmed runs will be measured.
- **Search efficiency:** Candidates generated, candidates surviving Stage 0, states expanded, repeated states avoided, and expansions needed to reach a verified result. A 40–60% expansion reduction on sufficiently branching tasks is a proposed research target, not an assumed outcome.
- **Result validity and quality:** PLN proofs will be replayed through PLN's own rules, truth-value calculations, and evidence checks. OpenPsi plans will be replayed through its transitions; simulated execution will additionally measure completed goals and failures. Invalid results are reported, not hidden by a success-rate aggregate.
- **Neural-policy generalization:** Compare in-domain and held-out performance against the symbolic policy and native control. Calibration and confidence on unfamiliar candidates will be checked alongside search outcomes, so a high-scoring but brittle model cannot appear successful merely from training-set results.
- **Scaling:** Plot success and end-to-end cost against graph size, branching factor, and proof or plan depth. Stage 0's effect on candidate generation will be measured separately from Tier 1's effect on ordering.
- **Exploratory-mode utility:** For tasks without a fixed goal, measure the number of novel, backend-verified, useful results found per compute budget against a predefined evaluation set or utility criterion. The criterion must be specified before comparing policies; "interesting" alone is not a reproducible metric.

8. Risks & Mitigations

Risk	Mitigation
Performance may vary across workloads. PRISM has delivered end-to-end speedups on several PLN tasks, but simple tasks and other components may not have enough search waste to offset control costs.	Measure time-to-valid-result and success under fixed budgets for each backend and task class. Cache or batch scores, avoid unnecessary Tier 2 calls, and let an adapter use native selection when the expected benefit of further control is too small
Stage 0 may remove a necessary candidate. A smaller frontier saves work, but a filter based on surface similarity could miss an early premise in a valid multi-step path.	Measure retention of known proof paths, test structurally diverse tasks, and widen or disable filtering when search stalls.
The neural policy may overfit. A model can score familiar templates quickly without learning guidance that transfers to new relations or domains.	Train on diverse backend-verified traces and require improvements in held-out, cross-domain *search outcomes* before using it in place of the symbolic policy.
Tier 2 may add excessive latency. An LLM-suggested subgoal can rescue a search under a step budget while taking longer in wall-clock time.	Invoke it only after a defined stall, limit calls and time per query, and evaluate its success gain alongside its full latency.
One policy may not suit every backend. Proof search, action planning, and evolutionary program search have different state spaces and notions of a useful result.	Reuse PRISM's candidate, budget, and trace interface while allowing backend-specific search policies and evaluation criteria.
Guidance may compromise result validity. A promising-looking connection is not necessarily a legal inference or executable plan; evidence must not be counted twice.	Have the backend apply and verify every accepted step, replay final proofs or plans, and block release on any invalid accepted result.
An integration failure may disrupt the component. Model loading, bridge calls, or strategic suggestions can fail independently of the backend.	Test predictable fallback to symbolic or native control, contain scorer errors, and return no result if authoritative backend verification fails.
Existing control methods may be competitive. Beating unguided search does not establish that PRISM adds value over native prioritization or ECAN attention	Compare all methods on identical tasks and budgets, including ECAN + PRISM, and report overhead and cases where PRISM loses.

10. Expected Contribution to iCog

PRISM would deliver a reusable search-control framework for Hyperon components: a common way to expose choices, allocate computation, learn from search traces, and evaluate control policies without replacing the components' own semantics.

Its initial research contribution is a rigorously evaluated neural-guided policy for uncertain inference, with PLN as the principal testbed and OpenPsi testing transfer to goal-directed planning. An ECAN integration will establish whether attention and explicit search control compete or complement one another. A subsequent MOSES adapter can test whether the framework extends to program search with a different policy.

The project would also leave iCog Labs with backend adapters, reproducible workloads, training traces, native and ECAN baselines, and evidence about where learned control helps, where it fails, and what it costs. That makes PRISM a continuing research and engineering component rather than a one-off improvement to a PLN example.

11. References

Geisweiller, N. *Meta-Reasoning in MeTTa for Inference Control*. Direct prior work on programmable search control in a MeTTa backward chainer. PRISM builds on this direction by developing learned guidance, reusable backend adapters, and cross-component evaluation.

Goertzel, B., et al. *OpenCog Hyperon: A Framework for AGI at the Human Level and Beyond*. Architectural context for PRISM as a component of the wider Hyperon research programme.

trueagi-io/PLN. The MeTTa PLN implementation and the authoritative source for the inference rules, truth-value calculations, and evidence handling used by the PLN adapter.

iCog Labs' PeTTa-OpenPSI. The OpenPsi port that provides the planning and action-selection backend for PRISM's second integration.

OpenCog Attention Allocation. Implementation reference for ECAN, relevant both as a general attention-allocation baseline and as a possible complementary signal for PRISM.

OpenCog AS-MOSES. Reference for the proposed program-search extension. It provides a materially different test of PRISM's reusable control interface from inference or action planning.

Silver, D., et al. *Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm*, and Hubert, T., et al. *Olympiad-level formal mathematical reasoning with reinforcement learning*. AlphaZero and AlphaProof provide precedents for combining learned guidance with search; AlphaProof further demonstrates search grounded in formally verified outcomes. They motivate PRISM's neural-guided search direction, without serving as a blueprint for its exact tiers.

Song, P., Yang, K., and Anandkumar, A. *Towards Large Language Models as Copilots for Theorem Proving in Lean*. Related work on learned proof-step suggestion, premise selection, and proof search in a formal system.

Goertzel, B. *Toward a Formal Model of Cognitive Synergy*. Broader motivation for improving how cognitive processes cooperate and allocate effort within an integrated architecture.

12. Appendix — Illustrative Scenario

Suppose the knowledge base contains uncertain relations that Sparrow inherits from Bird and Bird inherits from Animal. The query asks whether Sparrow inherits from Animal. The available search choices also include numerous unrelated relations and branches.

Stage 0 retrieves premises that could participate in a relevant inference. Tier 1 assigns a high priority to the candidate connecting the two inheritance relations. The search manager decides when to try it while retaining alternatives and respecting the budget. PLN then applies its own deduction rule, determines whether the premises are eligible, and computes any conclusion's truth value and evidence. PRISM neither supplies that conclusion nor calculates its truth value.

The evaluation question is whether this control process reaches the PLN-verified answer more reliably or cheaply than native, ECAN-guided, and other matched baselines—especially when distractors and depth increase.